# CMPSC 132 Final Project Report

## Tic-Tac-Toe Game

### Project Description

This project is a terminal-based Tic-Tac-Toe game implemented in Python. The game allows two players to play against each other by selecting positions on a 3×3 board. The board uses a number-based system (1–9) for input, which makes the interaction simpler and more user-friendly compared to row-column input.

The program includes two modes:

- Normal Mode: A single round of Tic-Tac-Toe.
- Competitive Mode: A best-of-three format where players compete until one player wins two rounds. A scoreboard is displayed after each round.

Additionally, the game uses emoji symbols (❌ and ⭕) instead of traditional X and O to improve readability and visual appeal.

### Approach and Logic

The program is structured using multiple helper functions to improve readability and maintain organization. Each function is responsible for a specific part of the game logic.

The board is initialized using a list of lists, where each inner list represents a row. Initially, the board contains string numbers from 1 to 9. When a player selects a position, the program converts that position into row and column indices using integer division and modulo operations.

The main gameplay occurs inside a loop that continues until a win or draw condition is reached. On each turn:

1. The board is displayed.
2. The player enters a position.
3. The program validates the input.
4. The move is applied if valid.
5. The program checks for a winner or draw.

The competitive mode builds on this logic by repeatedly calling a round function and keeping track of player scores until one player reaches two wins.

### Data Structures Used

The main data structure used in this project is a list of lists to represent the Tic-Tac-Toe board. This allows easy access and modification of specific positions using indices.

Other variables are used to track:

- The current player (❌ or ⭕)
- Player scores in competitive mode
- The current round number

This choice of data structures keeps the implementation simple while still being efficient and easy to manage.

**Challenges Faced**

One of the main challenges was handling user input safely. Since users can enter invalid values (such as letters or numbers outside the valid range), it was important to use exception handling (try/except) and validation checks to prevent the program from crashing.

Another challenge was mapping user input (positions 1–9) to the correct row and column in the board. This required understanding how to use mathematical operations to convert a single number into two indices.

Additionally, implementing the competitive mode required restructuring the game logic so that each round could return a result and update the scoreboard correctly.

**Possible Improvements**

There are several improvements that could be added in the future:

- Implementing an AI opponent so the user can play against the computer
- Adding a graphical user interface instead of a terminal-based interface
- Allowing players to enter custom names instead of using symbols

These enhancements would make the game more interactive and user-friendly.